Intrusion Detection with Machine Learning

Student Lab

Overview

In this lab, you will work with flow-like network records, train multiple classifiers, inspect their metrics, compare their behavior, and explain what the selected model output would mean for an intrusion detection workflow.

The goal is not to build a production IDS. The goal is to practice connecting a security task to data, model outputs, evaluation metrics, and human review.

Learning Goals

By the end of the lab, students should be able to:

- describe how network-flow features can support a simple intrusion detection task;
- train and evaluate several basic classifiers on labeled security data;
- compare at least three classification techniques and explain which worked best;
- compare a train/test score with five-fold cross-validation results;
- interpret false positives and false negatives as operational tradeoffs;
- explain what a human analyst would still need to review before acting on an alert.

Dataset

The starter package includes two public synthetic flow tables. Each row represents a connection summary rather than raw packets.

- `ids_flows_dev.csv`: a very small 30-row development set for inspection and discussion;
- `ids_flows_train.csv`: a 5,000-row training set for model development and cross-validation.

The instructor has a private 1,000-row withheld test set for bonus competition scoring. The withheld test set is not included in the starter package.

The fields are modeled after structured threat-detection logs. They include:

- timestamp, source and destination addresses, ports, protocol, packet and flow statistics, HTTP method, request path, and browser or tool user agent;
- security indicators such as failed login counts, IP reputation score, threat level, and traffic direction;

- a binary label indicating `benign` or `suspicious` activity;
- an attack category: normal traffic, brute force, port scan, data exfiltration, malware callback, or web attack.

The synthetic data is intentionally simple. It is appropriate for demonstrating workflow and tradeoffs, but it should not be presented as realistic network telemetry.

Dataset Difficulty

Synthetic security data can be made easy or hard to classify by design. A dataset becomes easier when one or two fields almost directly reveal the label, the classes are cleanly separated, the class balance is even, and there is little noise. A dataset becomes harder when benign and suspicious behavior overlap, labels are imperfect, suspicious events are rare, and the test data differs from the training data.

In this lab, you should look for signs that the dataset may be too clean. If a model performs very well, that does not automatically mean the model is ready for deployment. It may mean the synthetic labeling rule is easy to learn.

Feature Leakage Rule

Use `label` as the binary target. Do not use `label` as an input feature. Do not use `attack_category` as an input feature for the binary classifier because it reveals whether the row is normal or suspicious. You may use `attack_category` after prediction to analyze which categories were easier or harder to detect.

Your Task

Students complete a Colab notebook that asks them to:

1. load the flow dataset;
2. inspect class balance, attack categories, and feature distributions;
3. split the training data into training and validation sets;
4. create and justify derived features in code;
5. train at least three classifier types, choosing from reasonable supervised-learning options;
6. compare which classifier works best and explain why;
7. evaluate accuracy, precision, recall, and the confusion matrix;
8. run five-fold cross-validation to estimate whether the result is stable;
9. designate the best classifier in `best_model.py`;
10. adjust or discuss a decision threshold;
11. use attack categories to interpret which suspicious behaviors are easier or harder to detect;

12. write a short interpretation of what the results mean for IDS use.

The notebook contains TODOs in both code cells and markdown cells. Code TODOs affect whether the workflow runs. Markdown TODOs ask you to explain decisions and interpret results. Both kinds of work are graded as part of the total submission.

Classifier Comparison Requirement

Your submission must include performance results for at least three different classifiers. The first three classifiers should be different classifier types, such as logistic regression, a decision tree, a random forest, k-nearest neighbors, naive Bayes, gradient boosting, or a support vector machine. Do not count the same classifier with small parameter changes as three different classifiers.

For each classifier, report validation performance and five-fold cross-validation performance. Then identify one best classifier, explain why you selected it, and implement that selected pipeline in `best_model.py`.

Your comparison should discuss more than the final score. Consider whether the model is easy to explain, whether it handles nonlinear feature interactions, whether it may overfit, and whether its false positives or false negatives are acceptable for an IDS workflow.

Evaluation Notes

Complete `evaluation_notes.md` as the main written summary of your model evidence. This file is not a place to paste every notebook output. Instead, use it to copy the key metrics, identify the model settings, and explain what the results mean for intrusion detection.

The model comparison in `evaluation_notes.md` is organized as a list of classifier blocks rather than a table. Fill in one block for each classifier. For each block, include:

- the classifier name and important settings;
- validation F1 from the train/validation split;
- five-fold cross-validation mean F1;
- five-fold cross-validation F1 standard deviation;
- the model's main strength;
- the model's main weakness.

For each prompt in `evaluation_notes.md`, write your response on the indented line that begins `Answer here:`. Replace that placeholder with your own answer. Round numeric metrics to three decimals unless your instructor gives a different format.

`evaluation_notes.md` should also identify your selected classifier, summarize the selected model's accuracy, precision, recall, F1, cross-validation results, and confusion-matrix observation, and explain the security meaning of the observed false positives and false negatives.

Notebook And Submission Workflow

The notebook is your exploration and evidence file. Use it to inspect the data, create derived features, compare at least three classifiers, run validation and cross-validation, choose a threshold if useful, and justify your selected approach.

`best_model.py` is the reproducible scoring file. After you finish the notebook, transfer your final feature engineering and selected classifier into `best_model.py`. The instructor will run that file against withheld data. The feature engineering, selected classifier, threshold, and model name in `best_model.py` should match what you describe in the notebook, `evaluation_notes.md`, and `reflection.md`. Your `best_model.py` should define a complete pipeline through:

- `add_features(df)` for any derived features;
- `make_features(df, feature_columns=None)` for dropping leakage columns and encoding features;
- `train_best_model(train_df)` for fitting your selected model;
- `predict_best_model(model, metadata, test_df)` for returning 0/1 predictions.

AI Agent Use

You may use AI agents or assistants for debugging, explaining code, or brainstorming features and model choices. The starter package includes an `AGENTS.md` file that describes the intended boundaries for agent help. Work within those constraints when using an agent for this assignment. The point is to learn how the modeling choices, metrics, and security interpretation fit together, not to outsource the reasoning.

An agent may help you understand errors, compare possible approaches, or think through alternatives. You remain responsible for the final feature choices, classifier choices, written explanations, security interpretation, and submitted code. Record meaningful AI help in `ai_assistance_log.md`.

Questions To Answer

Complete these questions in `reflection.md`. Your answers may refer to specific metrics, confusion matrices, or model-comparison notes from your notebook and `evaluation_notes.md`. As in `evaluation_notes.md`, write each response on the indented line that begins `Answer here:` and replace the placeholder with your own answer.

- Which errors are more costly in this lab: false positives or false negatives?
- What would a human analyst need to see before acting on a model alert?
- What assumptions does the synthetic dataset make about attack behavior?
- How would your conclusions change if the dataset were much larger, noisier, or more imbalanced?
- Did the train/test result and cross-validation result tell the same story?
- Which classification technique worked best, and how does that connect to supervised learning concepts from Week 1?

- What would make a withheld challenge set fair for comparing different models?
- What additional evidence would you want before deploying this model in a security workflow?

Bonus Competition

Submit all three classifier results in your notebook and `evaluation_notes.md`. Also submit the starter file named `best_model.py`. In that file, implement your selected feature pipeline and best classifier. The instructor will run your selected model on a private withheld test set.

Bonus points are awarded by withheld-set ranking:

- top 5 submissions: 5 bonus points;
- top 2 submissions: 10 total bonus points;
- top 1 submission: 15 total bonus points.

Submission Package

Submit the following files:

- `ids_colab_explorer.ipynb`, completed as your exploration and evidence notebook, with code TODOs completed and notebook `Answer here:` placeholders replaced;
- `evaluation_notes.md`, completed with classifier blocks for at least three classifiers, validation performance, five-fold cross-validation performance, selected-model metrics, and security interpretation;
- `best_model.py`, completed with your final feature pipeline, selected classifier, threshold choice, and prediction logic;
- `incident_review.md`, completed with alert interpretation, false-positive and false-negative review, and escalation notes;
- `reflection.md`, completed with answers to the *Questions To Answer* section, including model-difference, pipeline-transfer, limitation, and human-review reflections;
- `ai_assistance_log.md`, if you used AI assistance.

Security Takeaway

A model result is not a security decision by itself. You should be able to explain what task the model supports, what evidence it uses, what its errors cost, and where human judgment remains necessary.

AI Development Acknowledgment

This lab was developed with assistance from AI tools. The instructor reviewed and revised the lab materials, datasets, code, and scoring workflow. If you notice unclear instructions, coding issues, unrealistic data behavior, or other rough edges, please report them so the lab can be improved.

Student-Facing Rubric

Category	Points	What strong work demonstrates
Security task framing	15	Clearly explains the IDS task and what the model output supports.
Data understanding	15	Identifies label balance, feature meaning, and synthetic-data limits.
Three-classifier comparison	15	Trains at least three classifier types and explains which performs best.
Train/test, cross-validation, and withheld-scoring preparation	25	Reports validation and cross-validation metrics, selects one best model, and prepares `best_model.py` for private scoring.
Human review and limitations	15	Explains what an analyst still needs to inspect or decide.
Communication	15	Presents conclusions clearly and makes a defensible security recommendation.